

Teaching Exam

Subject – Computer Science

Topic – Digital Logic



Lecture No. – 6

By – Rahul Sir

Topics to be Covered

Topic



- 1:- Universal Logic Gates ✓
- 2:- How to create basic Logic Gate using Universal Gate ✓
- 3:- Boolean Algebra ✓
- 4:- Laws of Boolean Algebra ✓
- 5:- Conclusion on Boolean Algebra ✓
- 6:- Canonical Form – SOP and POS ✓
- 7:- Difference between SOP and POS ✓



What is a Logic Gate?

A gate can be defined as a digital circuit that can allow a signal (electric current) to pass or stop. A type of gate that allows a signal to pass through when certain logical conditions are met. Different logic gates have different logical conditions.

Truth Table: A truth table is a table that shows all possible combinations of inputs and outputs for a logic gate.

Types of Gate

Given Below are the different types of Logic Gates :

- AND Gate(.)
- OR Gate(+)
- NOT Gate(')
- XOR Gate
- NAND Gate
- NOR Gate
- XNOR Gate
- Buffer Gate
- Universal Logic Gates



AND Gate(.)

The AND gate gives an output of 1 when if both the two inputs are 1, it gives 0 otherwise. For n-input gate if all the inputs are 1 then 1 otherwise 0. The AND gate operation is similar to the standard multiplication of 1s and 0s. The (.) dot represents the AND operation.

$$X = (A.B)$$

Where,

X is the output of gate

A and B are the inputs of the gate

2- Input AND Gate



Truth Table

A (Input 1)	B (Input 2)	X = (A.B)
0	0	0
0	1	0
1	0	0
1 ✓	1 ✓	1 ✓

AND Gate



OR Gate(+)

The OR gate gives an output of 1 if either of the two inputs are 1, it gives 0 otherwise. For n-input gate if all the inputs are 0 then 0 otherwise 1. The OR Operation is represented by the +.

2-Input OR Gate



The Expression for OR Gate is given as

$$X=A+B$$

Where,

X is the output of gate

A and B are the inputs of the gate

Truth Table

Input A	Input B	Output
0	0	0
0	1	1 ✓
1	0	1 ✓
1	1	1 ✓



NOT Gate(')

The NOT gate gives an output of 1 if the input is 0 and vice-versa. It is also known as Inverters. In Boolean algebra NOT operation is represented by bar over the variable such as $\bar{A}$.

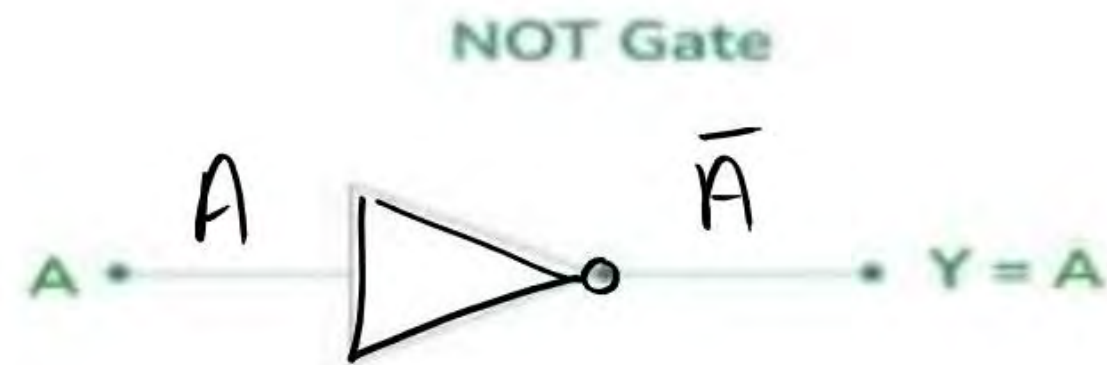
The Expression for NOT Gate can be given as

$$Y = \bar{A}$$

Where,

X is the output of gate

A is the inputs of the gate



Truth Table

A (Input)	Y = $\bar{A}$ (Output)
0 ✓	1 ✓
1 ✓	0 ✓

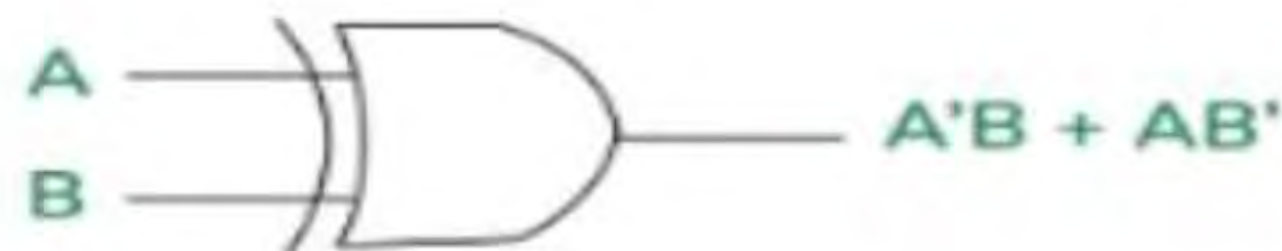
NOT Gate



XOR Gate

The XOR gate gives an output of 1 if either both inputs are different, it gives 0 if they are same. For n-input gate if the number of input 1 are odd then it gives 1 otherwise 0. For a two-input XOR gate it means that the output is true only if exactly one of the inputs is true.

XOR Gate



The Expression for XOR Gate can be given as

$$X = A'B + AB'$$

Where,

X is the output

A and B are the inputs

Truth Table

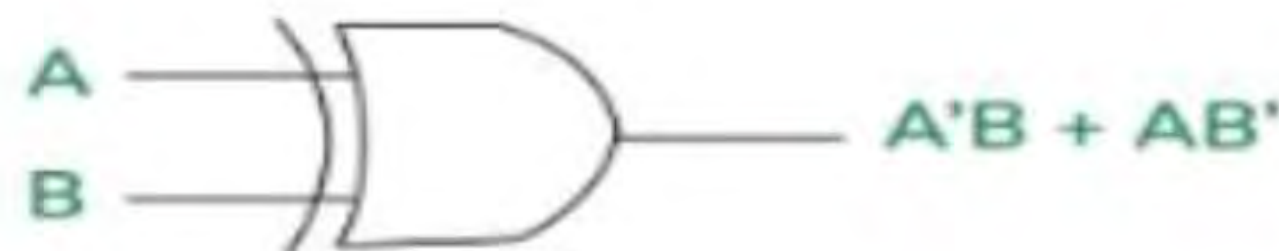
A (Input 1)	B (Input 2)	$X = A'B + AB'$
0	0	0
0	1	1
1	0	1
1	1	0



XOR Gate

The XOR gate gives an output of 1 if either both inputs are different, it gives 0 if they are same. For n-input gate if the number of input 1 are odd then it gives 1 otherwise 0. For a two-input XOR gate it means that the output is true only if exactly one of the inputs is true.

XOR Gate



The Expression for XOR Gate can be given as

$$X = A'B + AB'$$

$$\bar{A}B + A\bar{B}$$

Where,

X is the output

A and B are the inputs

Truth Table

A (Input 1)	B (Input 2)	$X = A'B + AB'$
0	0	0
0	1	1
1	0	1
1	1	0



NAND Gate

The NAND gate (negated AND) gives an output of 0 if both inputs are 1, it gives 1 otherwise. For n-input gate if all inputs are 1 then it gives 0 otherwise 1. The Term "NAND" can be said as "Not AND".

The Expression for NAND Gate can be given as

$$Y = \overline{A.B}$$

Where,

Y is the Output of gate

A and B is the input of gate

★
WHEN BOTH INPUT HIGH
THE OUTPUT IS LOW

2-Input NAND Gate



Truth Table

Input A	Input B	$X = (A.B)'$
0	0	1 ✓
0	1	1 ✓
1	0	1 ✓
1	1	0 ✗



NOR Gate

The NOR gate (negated OR) gives an output of 1 only if both inputs are 0, it gives 0 otherwise. For n-input gate if all inputs are 0 then it gives 1 otherwise 0. The "NOR" can be said as "NOT OR".

2- Input NOR Gate



The Expression for NOR Gate can be given as

$$Y = \overline{A + B}$$

Where,

Y is the Output of gate

A and B is the input of gate

★
WHEN BOTH INPUT LOW
THE OUTPUT IS HIGH

Truth Table

Input A	Input B	$0 = (A + B)'$
0	0	1
0	1	0
1	0	0
1	1	0

NOR Gate



XNOR Gate

The XNOR gate (negated XOR) gives an output of 1 if both inputs are same and 0 if they are different. For n-input gate if the number of input 1 are even then it gives 1 otherwise 0 or if the number of input 0 is even then the output is 1, otherwise 0'. The "XNOR" can be said as "Exclusive NOR".

What is XNOR Gate?



Truth Table

Input A	Input B	Output
0	0	1 ✓
0	1	0
1	0	0
1	1	1 ✓

XNOR Gate

$$Y = A \odot B$$

Where,

Y is the Output of gate

A and B is the input of gate



Buffer Gate

A Buffer Gate is a digital logic gate that amplifies a signal. It doesn't change the logic state (it outputs exactly what is input), but its primary purpose is to ensure that the signal can drive more circuits without weakening. Essentially, a buffer isolates the input from the output, making the output more robust.

- If the input is **1** , the buffer outputs **1** .
- If the input is **0** , the buffer outputs **0** .

Buffers are often used when a signal needs to drive a large number of other gates or when it needs to travel over a long distance without losing strength.

The Expression for Buffer can be given as

$$Y=A$$

Where,

Y is the Output of gate

A is the input of gate

Buffer



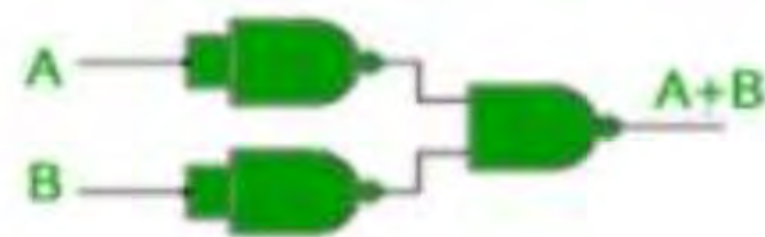
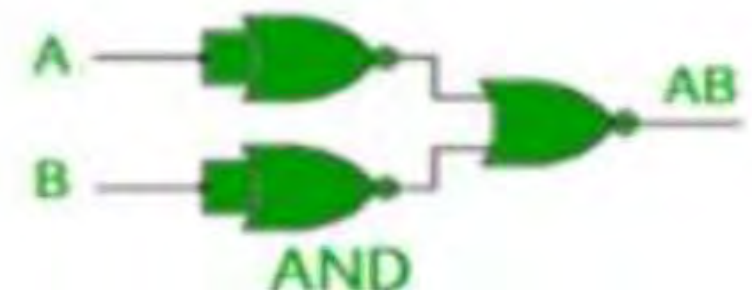
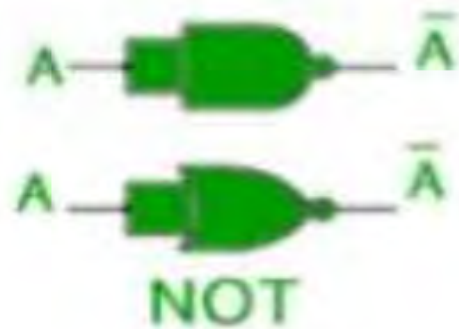
Truth Table

Input(A)	Output(Y)
0	0
1	1



Universal Logic Gates

Out of the eight logic gates discussed above, NAND and NOR are also known as **universal gates** since they can be used to implement any digital circuit without using any other gate. This means that every gate can be created by NAND or NOR gates only. Implementation of three basic gates using NAND and NOR gates is shown below -

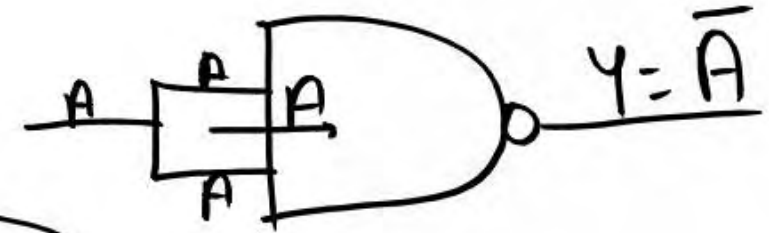
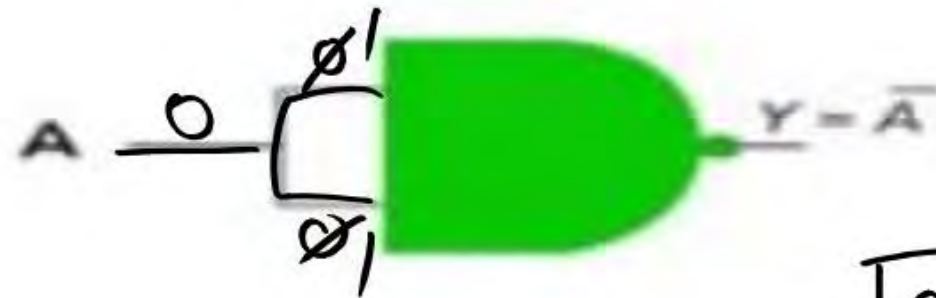




NOT Gate (Inverter) ✓

Circuit: Connect both inputs of a NAND gate to the same input signal.

NOT Gate using NAND Gate



$$A \cdot A = A$$

Truth Table

A (Input)	B (Input)	$Y = (\overline{AB}) = \bar{A}$
0	0	1
1	1	0

NOT GATE USING NAND GATE



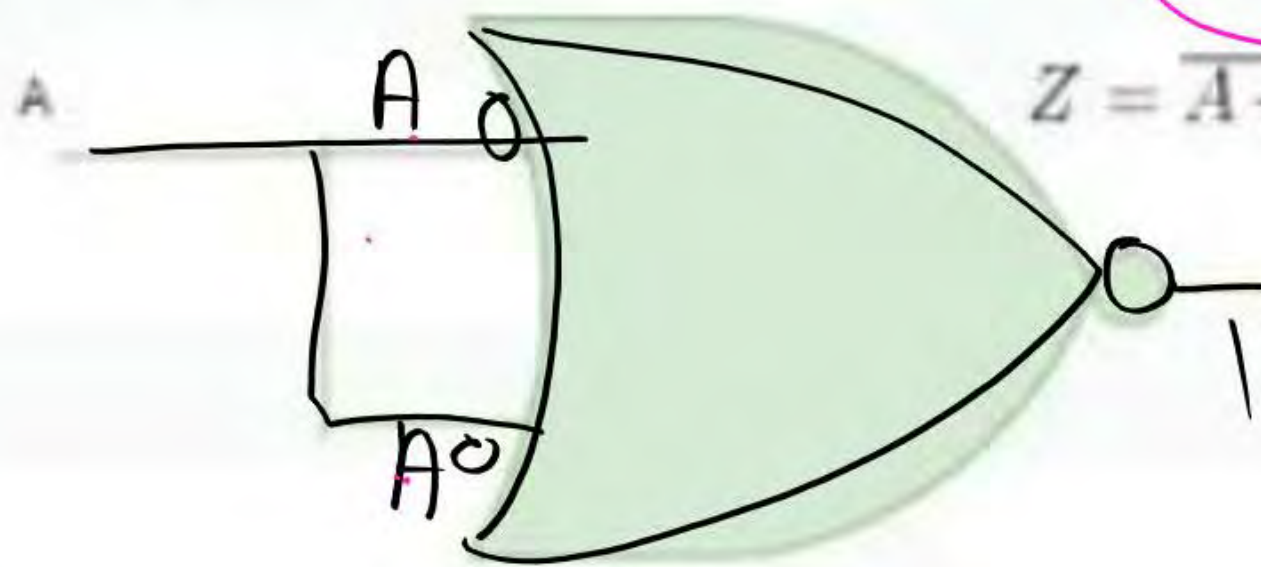
NOT Gate (Inverter)

Circuit: Connect both inputs of a NOR gate to the same input signal.

Truth Table

A	Output (NOT A)
0	1
1	0

$$\checkmark \underline{A + A = A}$$
$$\checkmark \underline{A \cdot A = A}$$



$$Z = \overline{A + A} = \overline{A}$$

$$\overline{A + A} \Rightarrow \overline{A}$$

$$\overline{A + A} \Rightarrow \overline{A}$$

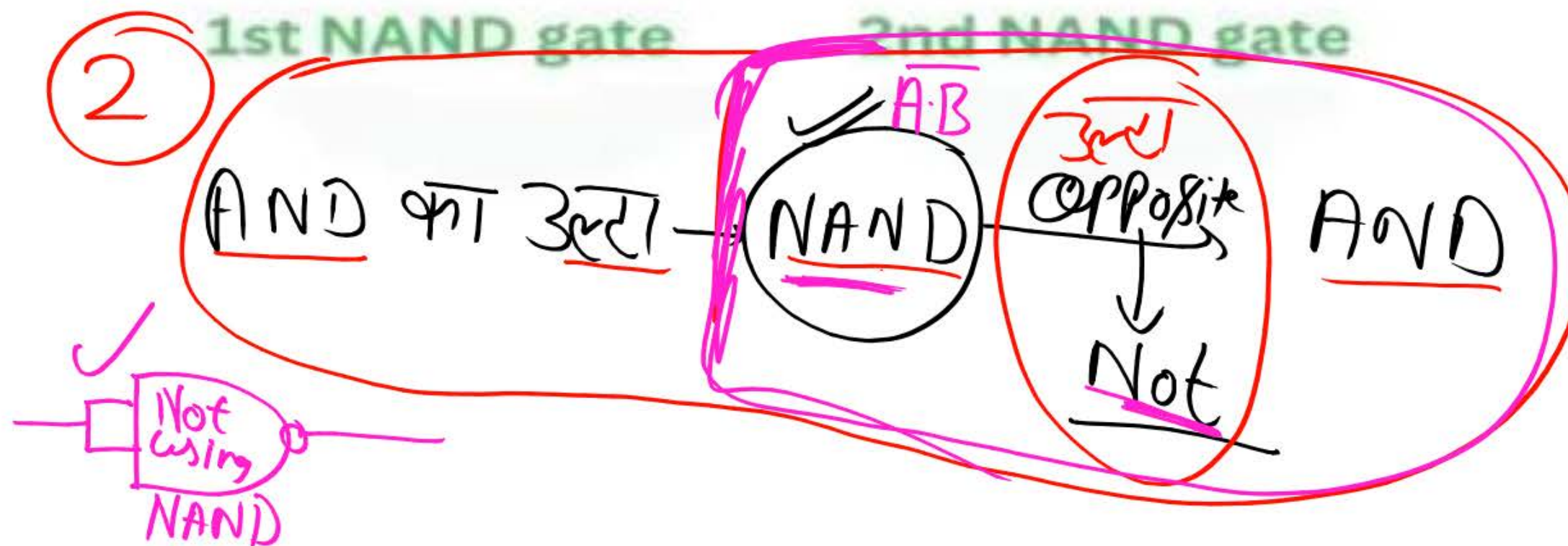
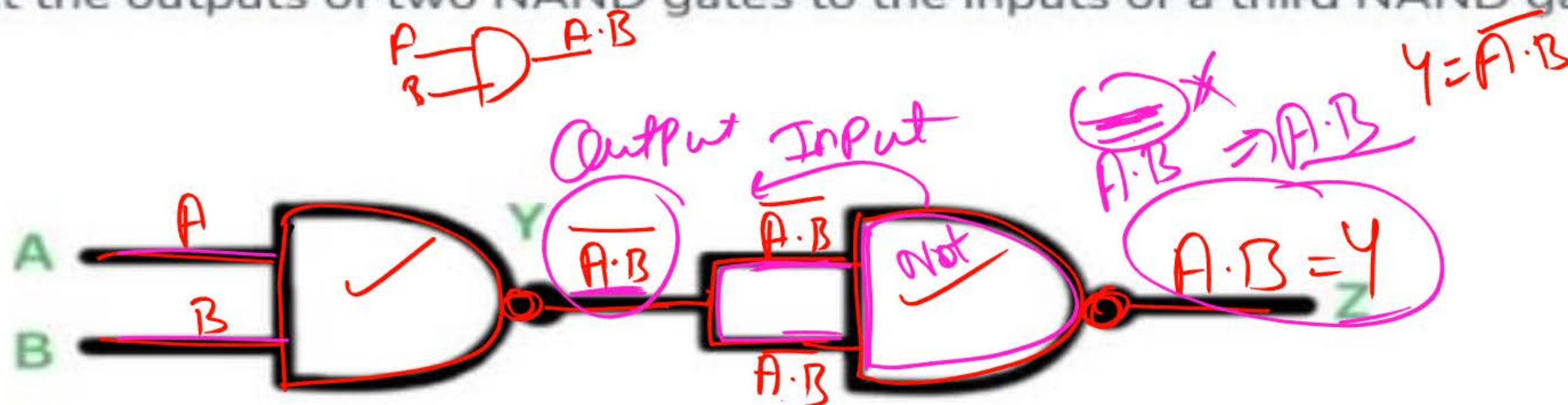


AND Gate

Circuit: Connect the outputs of two NAND gates to the inputs of a third NAND gate.

Truth Table

A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

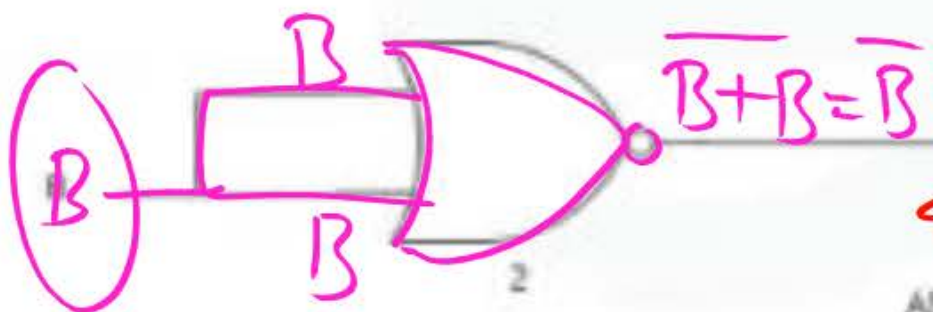
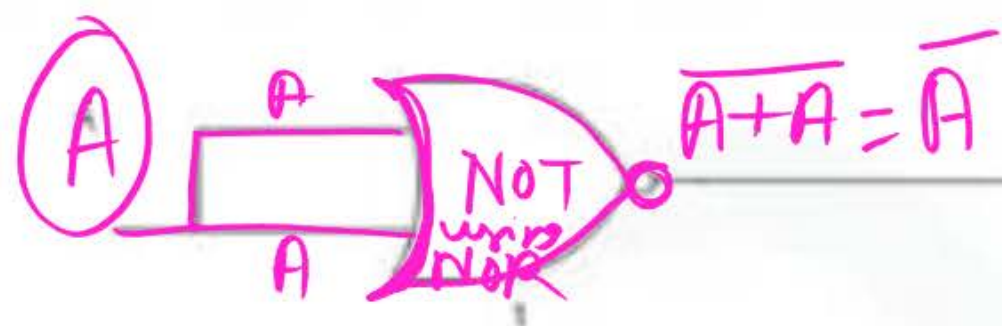




AND Gate



Circuit: Use three NOR gates. First, connect each input to a NOR gate and then connect their outputs to the inputs of another NOR gate.



$$\overline{A+B}$$

$$\overline{A} \cdot \overline{B}$$

$$A \cdot B$$

$$\overline{A}$$

$$\overline{B}$$

$$\overline{A+B}$$

$$\overline{A} \cdot \overline{B}$$

$$A \cdot B$$

AND

3 NOR

2 NAND

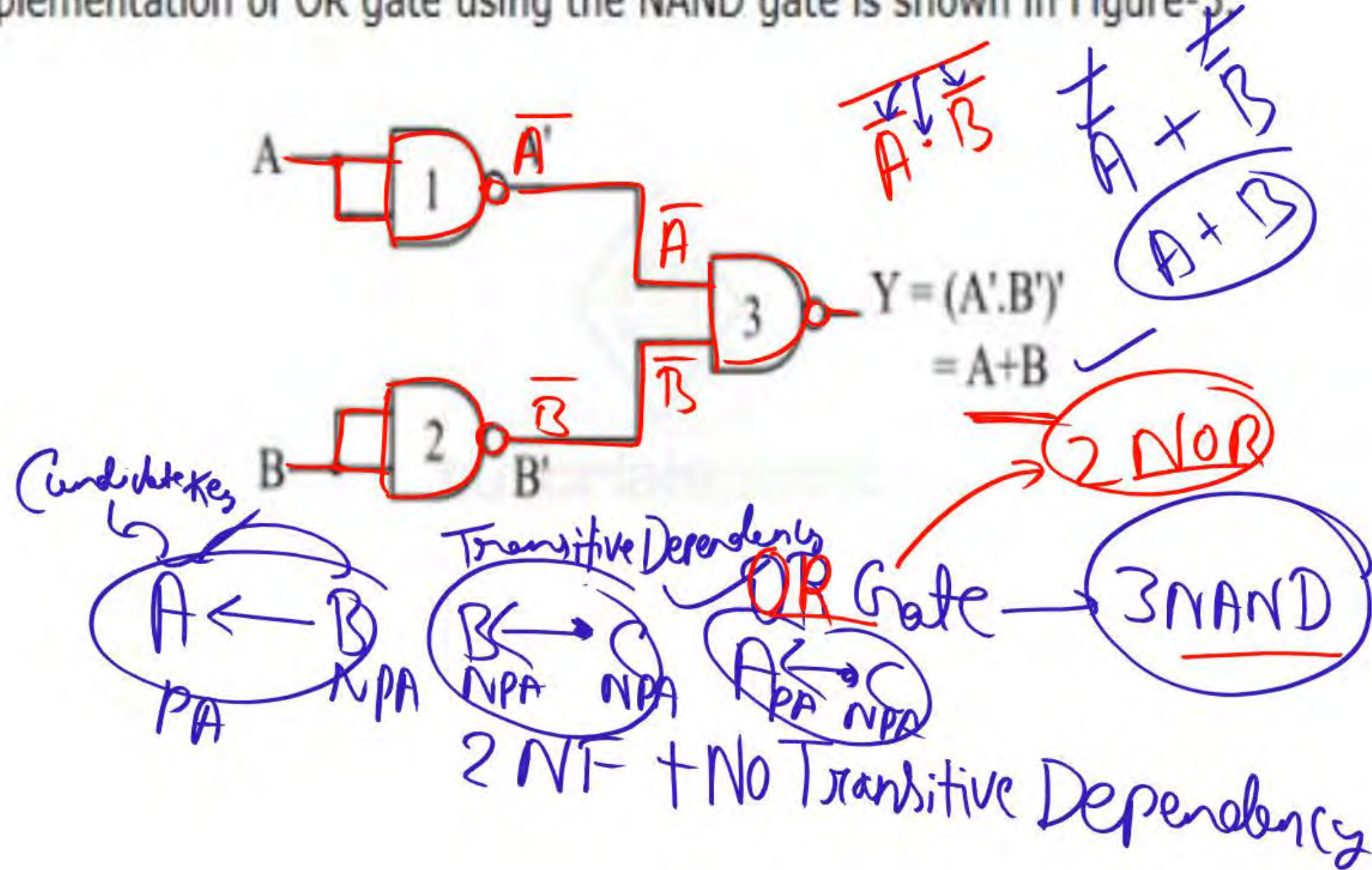
Truth Table

Starting I ₁ I ₂		final output AND	
A	B	Output (A AND B)	
0	0	0	
0	1	0	
1	0	0	
1	1	1	



OR Gate

The NAND gate is a universal gate, therefore, it can be used to realize the OR gate.
The implementation of OR gate using the NAND gate is shown in Figure-3.



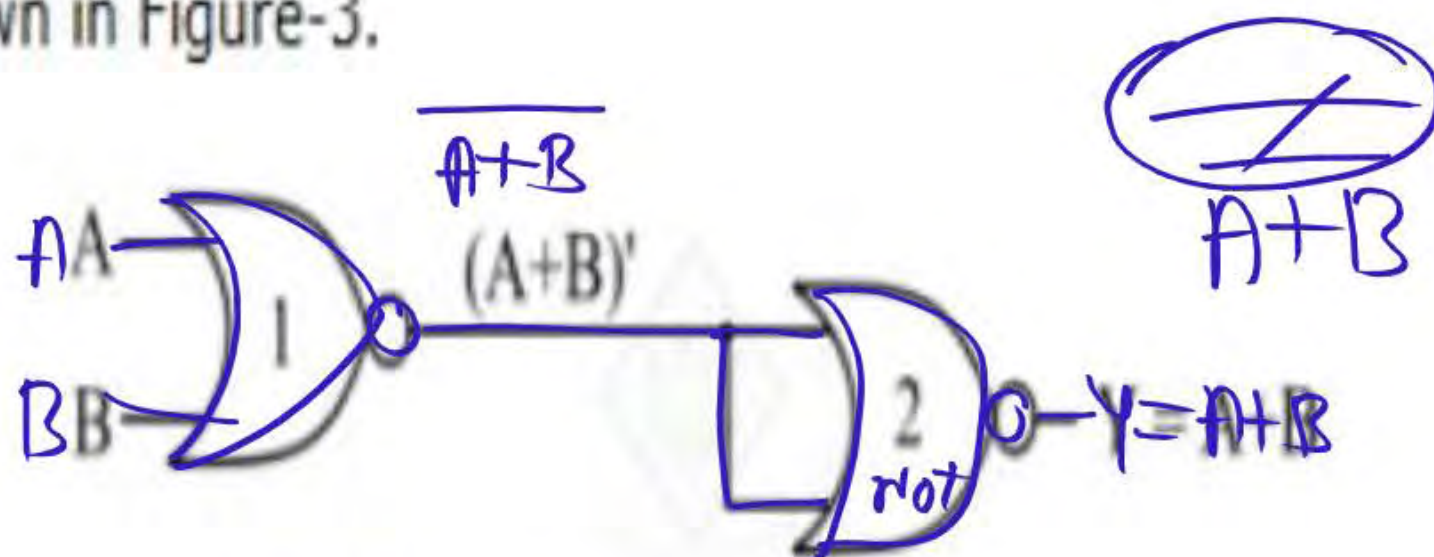
Truth Table

A	B	Output (A OR B)
0	0	0
1	0	1
0	1	1
1	1	1



Implementation of OR Gate from NOR Gate

As we know, the NOR gate is a type of universal logic gate, therefore, using NOR gates only, we can implement the OR operation. The logic diagram of OR Gate using NOR Gate is shown in Figure-3.



OR $\rightarrow$ 2 NOR
 $\rightarrow$ 3 NAND

Truth Table

A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0



4 gate NAND
NAND using NOR
NOR using NAND
4 gate NOR

Learn

	NAND	NOR
<u>NOT</u>	1	1
<u>AND</u>	2	3
<u>OR</u>	3	2
<u>XOR</u>	4	5
<u>XNOR</u>	5	4



Fundamental Laws & Theorems of Boolean Algebra

The fundamental laws and theorems of Boolean algebra simplify logical expressions and help in designing digital circuits. Some of the core laws and theorems in Boolean algebra are:

1. Commutative Law

The Commutative Law states that the order in which two variables are combined using the AND or OR operators does not affect the result. This law allows us to reorder terms without changing the output.

$$A + B = B + A$$

- Commutative Law for AND:

$$A \cdot B = B \cdot A$$

$$A \cdot B = B \cdot A$$

Example:

$$1 \cdot 0 = 0 \cdot 1 = 0$$

- Commutative Law for OR:

$$A + B = B + A$$

Example:

$$1 + 0 = 0 + 1 = 1$$

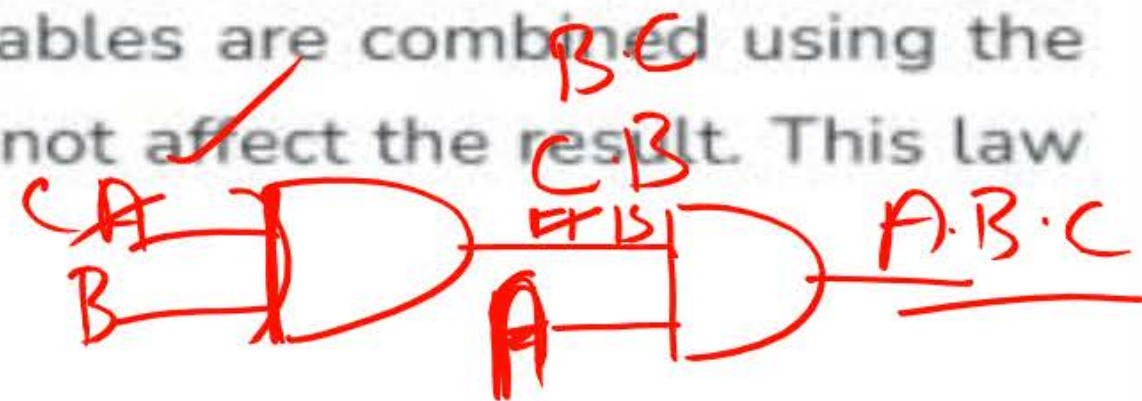


2. Associative Law

The Associative Law states that when three or more variables are combined using the AND or OR operators, the grouping of the variables does not affect the result. This law allows us to regroup terms without changing the output.

$$A + \overline{B \cdot C}$$

$$(A + \overline{B} + \overline{C})$$



- Associative Law for AND:

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$

Example:

$$1 \cdot (0 \cdot 1) = (1 \cdot 0) \cdot 1 = 0$$

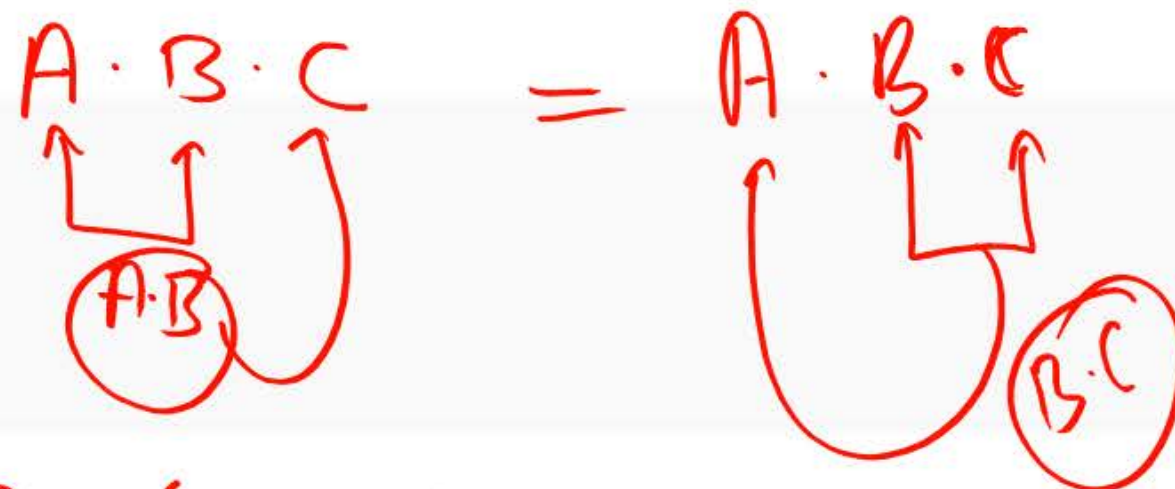


- Associative Law for OR:

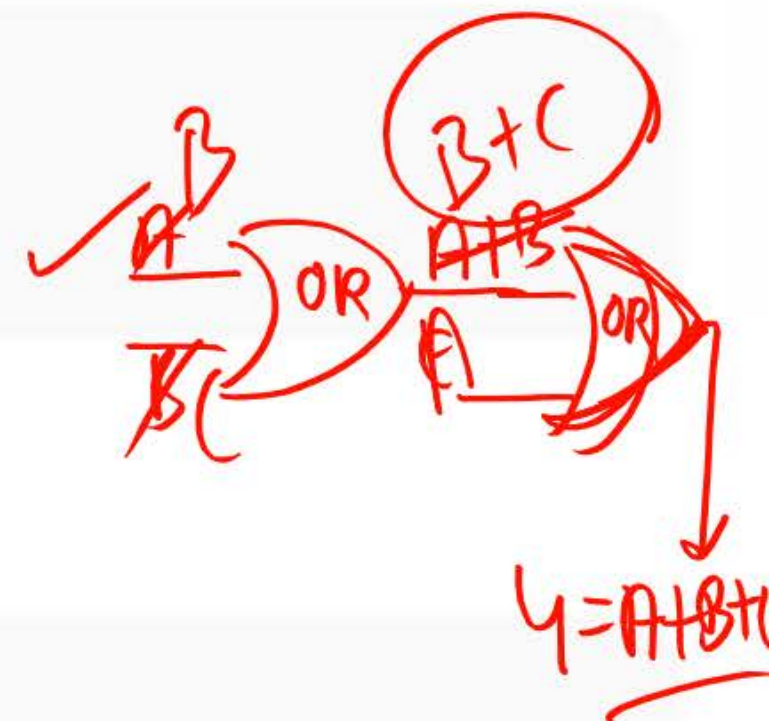
$$A + (B + C) = (A + B) + C$$

Example:

$$1 + (0 + 1) = (1 + 0) + 1 = 1$$



$$A + (B + C) = (A + B) + C$$





3. Distributive Law

The Distributive Law describes how the AND and OR operations distribute over each other. It is similar to how multiplication distributes over addition in arithmetic. This law allows the factoring of Boolean expressions, similar to factoring algebraic expressions.

- Distributive Law for AND over OR:

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$

Example:

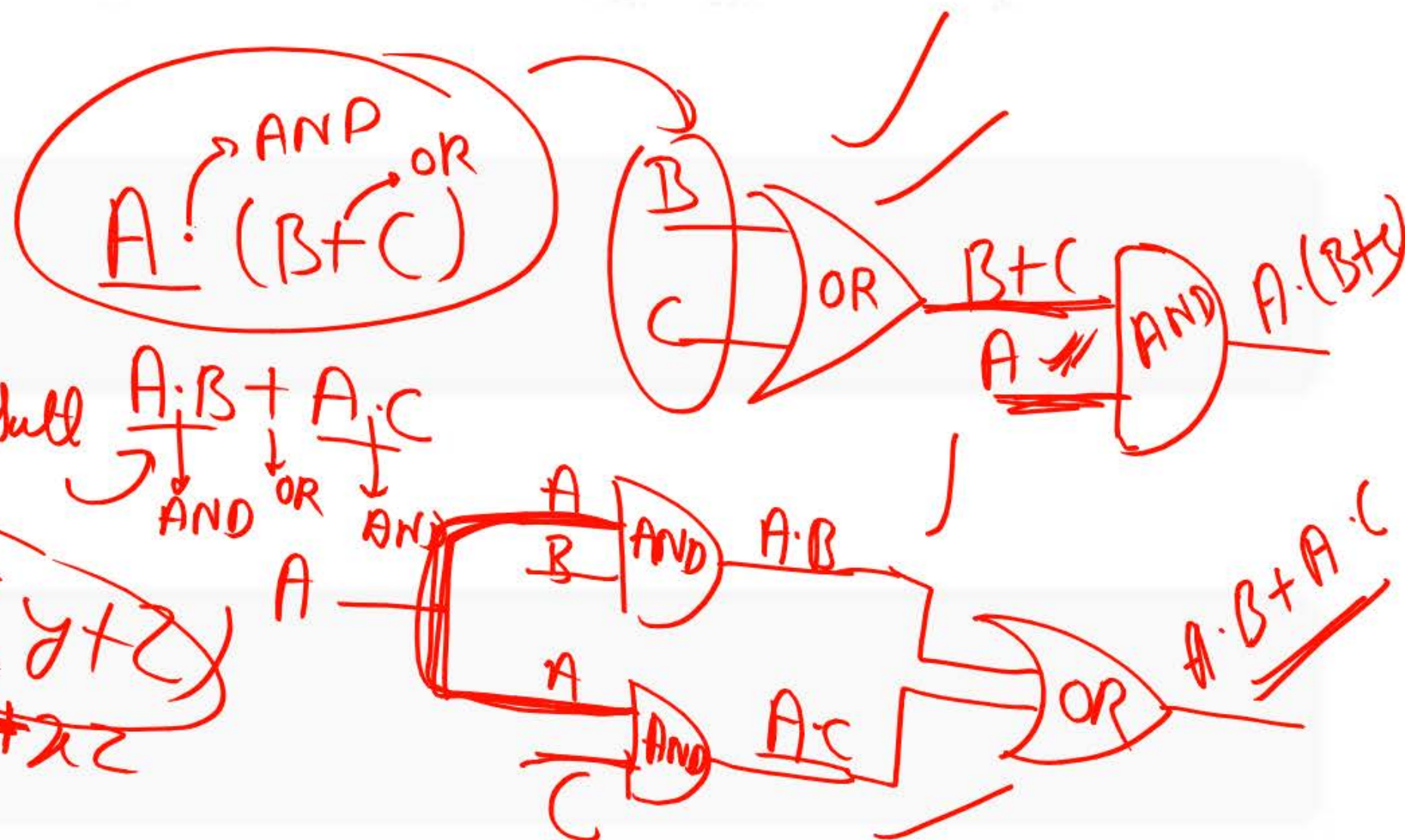
$$1 \cdot (0 + 1) = (1 \cdot 0) + (1 \cdot 1) = 0 + 1 = 1$$

- Distributive Law for OR over AND:

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

Example:

$$1 + (0 \cdot 1) = (1 + 0) \cdot (1 + 1) = 1 \cdot 1 = 1$$





4. Identity Law

The Identity Law states that any variable ANDed with 1 or ORed with 0 will result in the original variable itself. This law shows that the identity elements for AND and OR operations are 1 and 0, respectively.

• Identity Law for AND:

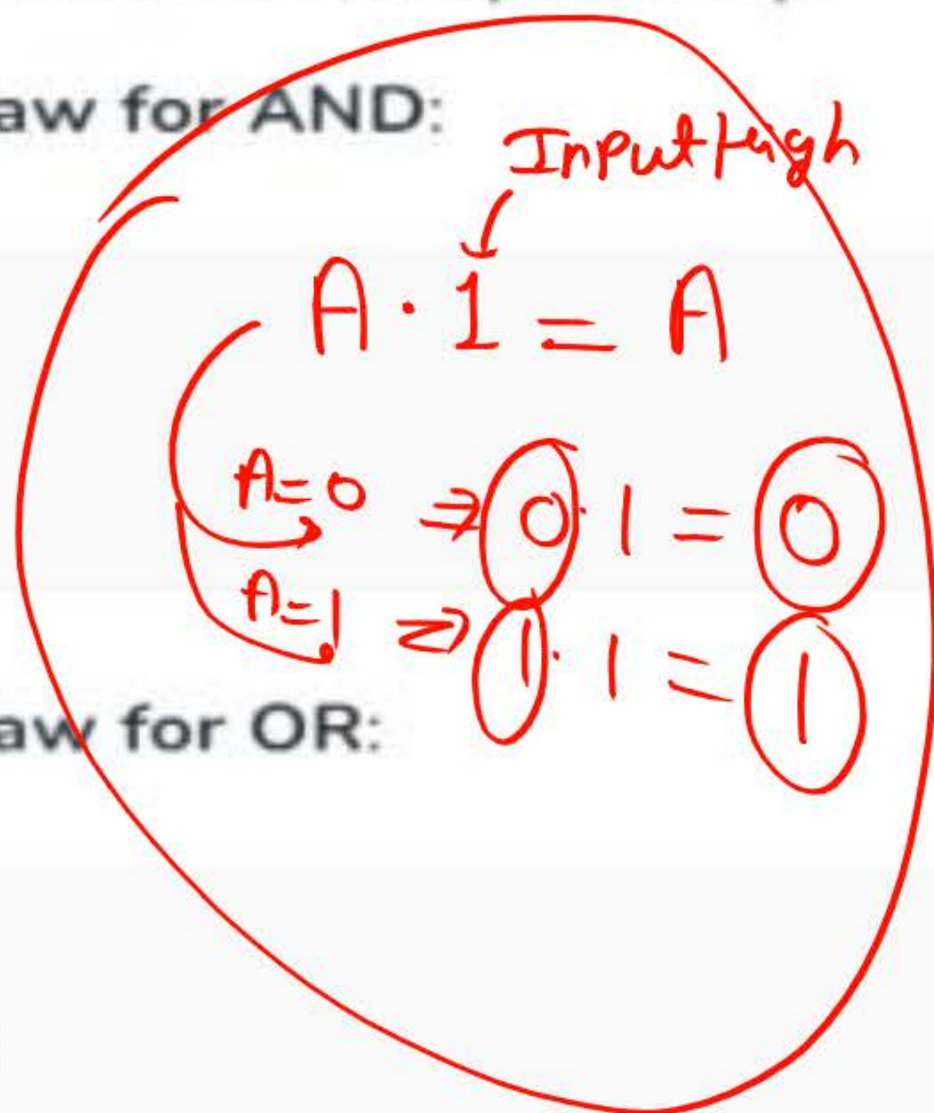
$$A \cdot 1 = A$$

Example:
 $1 \cdot 1 = 1$

• Identity Law for OR:

$$A + 0 = A$$

Example:
 $1 + 0 = 1$



Identity Law

$$\begin{aligned} A \cdot 1 &= A \\ A \cdot 0 &= 0 \\ A + 0 &= A \\ A + 1 &= 1 \end{aligned}$$

$1 + B$
 $1 + BC$

$$\begin{aligned} 1 + BC &= 1 \\ 1 + A + B + C &= 1 \\ 1 + B + C &= 1 \\ 1 + 0 &= 1 \end{aligned}$$

$1 + A + B + C \geq 1$



5. Complement Law

The Complement Law involves the negation of a variable and provides the result when a variable is combined with its complement (opposite). This law shows that a variable ANDed with its complement will always be 0 and a variable ORed with its complement will always be 1.

- Complement Law for AND:

$$A \cdot A' = 0$$

Example:

$$1 \cdot 1' = 1 \cdot 0 = 0$$

- Complement Law for OR:

$$A + A' = 1$$

Example:

$$1 + 1' = 1 + 0 = 1$$

$$A \cdot A' = A \cdot \bar{A} = 0 \Rightarrow 1 \cdot 0 \Rightarrow 0 \quad A=1$$

$$A + \bar{A} = 1 \Rightarrow 1 + 0 = 1$$

$$A=1$$

$$A=0$$

$$\bar{A} \cdot 0 = 0$$

$$0 \cdot 0 = 0$$

$$1 \cdot 0 = 0$$



6. Inversion Law

The Inversion Law is a unique principle in Boolean algebra, stating that the complement of the complement of any variable is equal to the variable itself.

$$(A')' = A$$

Example:

$$(1')' = (0)' = 1$$

$\overset{A}{\curvearrowright} \bar{A} \Rightarrow$
inverse

~~(((A)))~~



6. Inversion Law

The Inversion Law is a unique principle in Boolean algebra, stating that the complement of the complement of any variable is equal to the variable itself.

$$(A')' = A$$

Example:

$$(1')' = (0)' = 1$$



Note: $\overline{-} \rightarrow +$
 $\overline{+} \rightarrow \cdot$

7. De Morgan's Theorems

De Morgan's Theorems provide a way to simplify expressions involving negations and are very useful in digital circuit design.

De Morgan's First Theorem: The negation of an AND operation is equal to the OR operation of the negations of the operands.

$$(A \cdot B)' = A' + B'$$

Example:

Expression: $(1 \cdot 0)'$

Simplified: $1' + 0' = 0 + 1 = 1$

Handwritten diagram for De Morgan's First Theorem:

$$\overline{A \cdot B} \rightarrow \overline{A} + \overline{B}$$

Handwritten diagram for De Morgan's First Theorem:

$$\overline{A + B} \rightarrow \overline{A} \cdot \overline{B}$$

Handwritten diagram for De Morgan's First Theorem:

$$\overline{BC}$$

De Morgan's Second Theorem: The negation of an OR operation is equal to the AND operation of the negations of the operands.

$$(A + B)' = A' \cdot B'$$

Example:

Expression: $(1 + 0)'$

Simplified: $1' \cdot 0' = 0 \cdot 1 = 0$

Handwritten diagram for De Morgan's Second Theorem:

$$\overline{A + BC} \rightarrow \overline{A} \cdot \overline{BC}$$

Handwritten diagram for De Morgan's Second Theorem:

$$\overline{A} \cdot BC \rightarrow \overline{A} \cdot B \cdot C$$



$$A + A = A$$

$$A \cdot A = A$$

Idempotent Law

$$A \cdot 1 = A$$

$$A \cdot 0 = 0$$

$$A \cdot A = A$$

$$(BC + BC) * (BC + A)$$

$$BC * (BC + A)$$

$$(BC * BC) + (A * BC)$$

$$BC + A * BC$$

$$BC (1 + A)$$

$$BC + (BC * A)$$

$$BC$$

$$A + 1 = 1$$

$$A + 0 = A$$

$$A + \textcircled{A} = A$$

$$1 + B = 1$$

Absorption

$$A + (A * B) = A$$

Absorption

$$(A + A) * (A + B)$$

$$A * (A + B)$$

$$A * A + A * B$$

$$A + A * B$$

$$A (1 + B)$$

$$A \cdot 1$$

$$A$$

$$A + (A * B) = A$$

$$A + (A * \text{Something}) = A$$

$$A * (A + B) = A$$

$$A * (A + \text{Something}) = A$$

$$(A * B) + (A * \sim B) = A$$

$$BC * (BC + A)$$

$$(A + B) * (A + \sim B) = A$$

$$\Rightarrow BC$$

$$A + (\sim A * B) = A + B$$

$$A * (\sim A + B) = A * B$$



Basic Laws of Boolean Algebra

In all the cases given below A can either be 0 or 1

1. $A \cdot 0 = 0$

2. $A \cdot 1 = A$

3. $A \cdot A = A$

4. $A \cdot \bar{A} = 0$

5. $A + 0 = A$

6. $A + 1 = 1$

7. $A + \bar{A} = 1$

8. $A + A = A$



Conclusion

Boolean algebra laws are a series of laws and theorems that help understand why or how something happens with Boolean algebra, which is essential to any computer program.

The main goal of logic design is to simplify the logic as much as possible so that the final implementation is easy. To simplify the logic, you need to simplify the Boolean equations and expressions that represent the logic. Some laws and theorems have been proposed to simplify the Boolean equations. Using these laws and theorems makes it very easy to simplify or reduce the logical complexity of a Boolean expression or function.



In Boolean algebra, SOP (Sum of Products) and POS (Product of Sums) are two canonical forms used to represent Boolean expressions. SOP is a logical sum (OR) of product terms (AND), while POS is a logical product (AND) of sum terms (OR). These forms are derived from truth tables and provide standardized ways to express Boolean functions.



SOP Vs POS in Digital Logic

Representation of Boolean expression can be primarily done in two ways. They are as follows:

1. Sum of Products (SOP) form
2. Product of Sums (POS) form

NOTE

If the number of input variables are n , then the total number of combinations in [Boolean algebra](#) is 2^n .

If the input variable (let A) value is :

- Zero (0) - a is LOW -It should be represented as A' (Complement of A)
- One (1) - a is HIGH -It should be represented as A



In Boolean logic,

AND is represented as '.'

A AND B is written as 'A.B'

OR is represented as '+'

A OR B is written as 'A+B'

For example, Considering number of input variables =3, Say A, B and C.

Total number of combinations are: $2^3=8$.



What is SOP?

It is one of the ways of writing a Boolean expression. As the name suggests, it is formed by adding (OR operation) the product terms. These product terms are also called as 'min-terms'. Min-terms are represented with 'm', they are the product(AND operation) of Boolean variables either in normal form or complemented form.



A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

The function X can be written in SOP form by adding all the min-terms when X is HIGH(1).

While writing SOP, the following convention is to be followed:

If variable A is Low(0) - A'
A is High(1) - A

$$\begin{aligned} X(\text{SOP}) &= \sum_m(1, 3, 6) \\ &= A'.B'.C + A'.B.C + A.B.C' \end{aligned}$$



Find the Expression :-

Q1:- $f(x,y,z) = \sum(0,4,6)$

Q2:- $f(x,y,z) = \sum(1,5,7)$

Q3:- $f(x,y,z) = \sum(1,5)$



What is POS?

As the name suggests, it is formed by multiplying (AND operation) the sum terms. These sum terms are also called as 'max-terms'. Maxterms are represented with 'M', they are the sum (OR operation) of Boolean variables either in normal form or complemented form.



Consider a function X, whose truth table is as follows:

A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

The function X can be written in POS form by multiplying all the max-terms when X is LOW(0).

While writing POS, the following convention is to be followed:

If variable A is Low(0) - A

A is High(1) - A'

$$X \text{ (POS)} = \prod_M(0, 2, 4, 5, 7)$$

$$= (A+B+C) \cdot (A+B'+C) \cdot (A'+B+C) \cdot (A'+B+C') \cdot (A'+B'+C')$$



Find the Expression :-

$$\text{Q1:- } f(x,y,z) = \prod(0,4,6)$$

$$\text{Q2:- } f(x,y,z) = \prod(1,5,7)$$

$$\text{Q3:- } f(x,y,z) = \prod(1,5)$$



S.No.	SOP	POS
1.	A way of representing Boolean expressions as sum of product terms.	A way of representing Boolean expressions as product of sum terms.
2.	SOP uses minterms. Minterm is product of Boolean variables either in normal form or complemented form.	POS uses maxterms. Maxterm is sum of Boolean variables either in normal form or complemented form.
3.	It is sum of minterms. Minterms are represented as 'm'	It is product of maxterms. Maxterms are represented as 'M'
4.	SOP is formed by considering all the minterms, whose output is HIGH(1)	POS is formed by considering all the maxterms, whose output is LOW(0)
5.	While writing minterms for SOP, input with value 1 is considered as the variable itself and input with value 0 is considered as complement of the input.	While writing maxterms for POS, input with value 1 is considered as the complement and input with value 0 is considered as the variable itself.

THANK YOU